LLM и NLP

Агенты

github.com/ivtipm/ML



Ветров С. В.

Содержание

1. Инструменты
2. Поиск в сети
3. Команды и код
4. Агенты

См. также
Prompt engineering

Доска с классификацией техник промптинга: excalidraw, png

Основы работы с LLM
docs.google.com/presentation/d/1DMAws01QZB-uQC1vCTuASi0T6tIjxIIH5OQGH9mFHmY

Слайды про модели типа Seq2Seq и Трансформеры
docs.google.com/presentation/d/1Kpk3DcHNf1iOMcggF701VorNuI3csqcuU9G69mUxtZU

Инструменты



Инструменты

Tool — An LM-used tool is a function interface to a computer program that runs externally to the LM, where the LM generates the function calls and input arguments in order to use the tool

Инструмент — это функциональный интерфейс к компьютерной программе, выполняемой вне LLM; модель генерирует текст с вызовами функций и аргументами, чтобы воспользоваться инструментом

У инструмента есть: имя (name), описание (description), схема параметров (JSON Schema)

синонимы инструмента: function calling, tool calling, tool use

Инструменты

Не все модели поддерживают инструменты

*На сайте ollama модели с поддержкой вызова инструментов помечены тегом **tools***

Модели нужно специально обучать вызову и работе с результатами работы инструментов.

Некоторые модели специализируются только на этом

nemotron-cascade-2

An open 30B MoE model from NVIDIA with 3B activated parameters that delivers strong reasoning and agentic capabilities.

tools thinking 30b

151.2K Pulls 3 Tags Updated 6 months ago

cogito-2.1

The Cogito v2.1 LLMs are instruction tuned generative models. All models are released under MIT license for commercial use.

671b

225.1K Pulls 5 Tags Updated 10 months ago

functiongemma

FunctionGemma is a specialized version of Google's Gemma 3 270M model fine-tuned explicitly for function calling.

tools 270m

195.8K Pulls 4 Tags Updated 9 months ago

LLM — генератор текста, он не умеет точно считать, выполнять код, получать свежие данные

Параметрическая память фиксирована — содержит знания на момент обучения.

Контекст ограничен окном.

Инструменты дают модели доступ к внешнему миру: свежие данные, вычисления, действия

Инструменты обеспечивают **grounding** — опору ответа на проверяемые внешние данные (снижение галлюцинаций)

Пример:
«Какая сейчас погода в Москве?»
модель не знает, но может вызвать API погоды

Вызов vs исполнение

LLM только решает какой инструмент вызвать и с какими параметрами, внешняя система выполняет действия инструмента

Это реализация идеи «интеллект + инструменты».

Инструменты позволяют получают ***точные результаты***, когда это необходимо (например вычисления) и ***совершать действия***

Такой подход также называют «LLM as a controller» (LLM как контроллер). Модель планирует задачу и делегирует подзадачи специализированным моделям/API

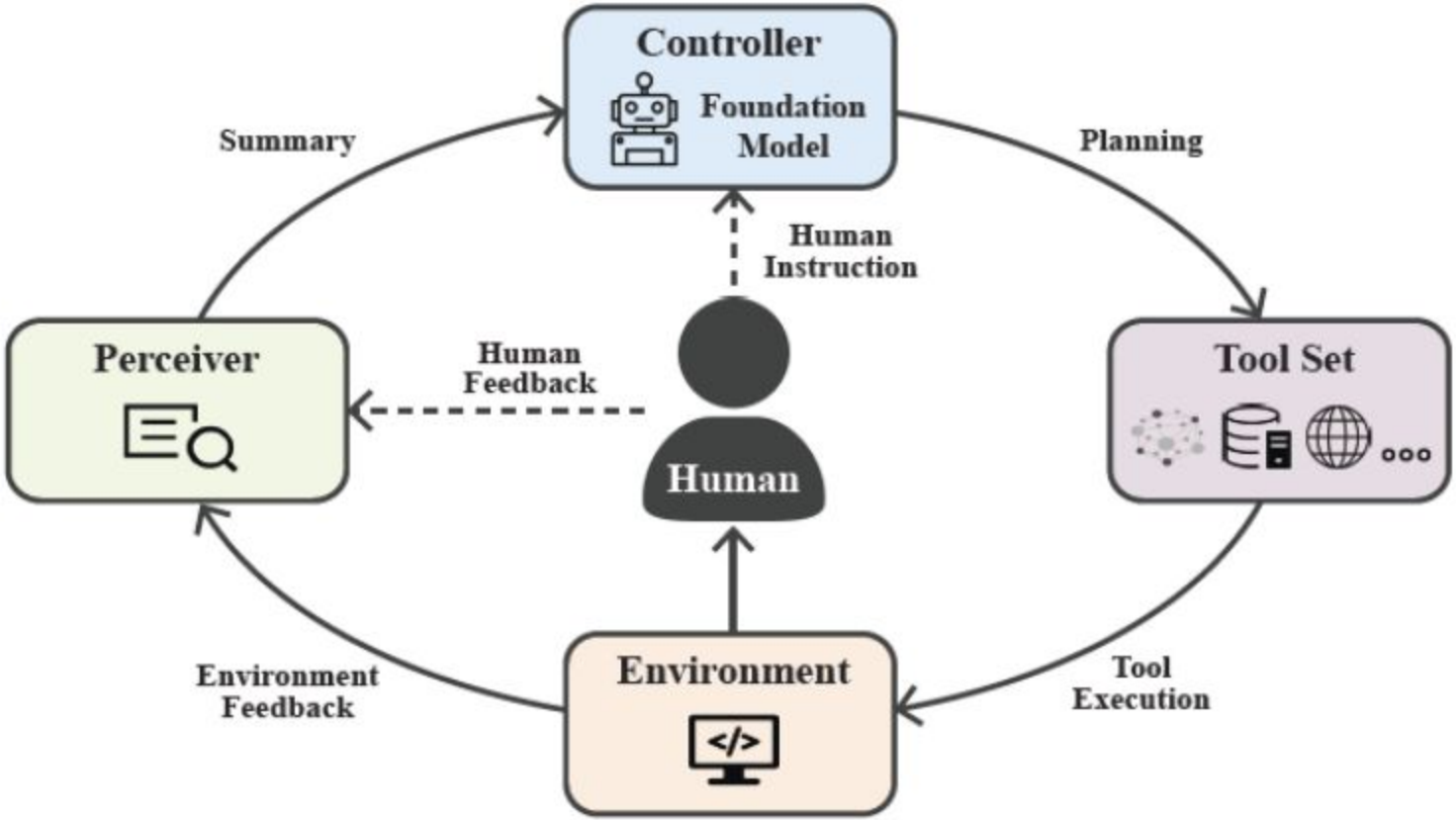
Что такое MVC?

Подходы

- 2022 — ReAct (Yao et al., Google): цикл **«мысль → действие → наблюдение»**; действия могут быть вызовами инструментов
- Февраль 2023 — Toolformer (Schick et al., Meta AI). **Модель сама учится решать**, когда вызывать инструменты

Инструменты

- Март 2023 — ChatGPT Plugins, Web browsing, Code Interpreter
- Июнь 2023 — OpenAI function calling — первый публичный API
- Ноябрь 2023 — Claude 2.1 tool use (Anthropic, бета; GA — май 2024)
- Июль 2024 — нативный tool calling в открытых моделях (Llama 3.1, Qwen, Mistral)
- Ноябрь 2024 — MCP (Model Context Protocol) — открытый стандарт подключения инструментов


Controller
Foundation Model
Summary
Planning
Human Instruction
Perceiver
Human Feedback
Human
Tool Set
Environment Feedback
Environment
Tool Execution

Эволюция подходов: как научить модель вызывать

Промпт-инжиниринг. «Выведи JSON с вызовом инструмента» — работает, но ненадёжно. Модель может выдать ответ не соответствующий формату

Дообучение (SFT). Модель учат выдавать вызовы в специальном формате (обычно JSON). Надёжность генерации текста с вызовом существенно повышается.

Используются **специальные токены** `<tool_call> / <tool_response>`
Модель с их помощью «переключается» между обычным текстом и текстом с вызовом инструмента.

Zero-shot tool use современные модели вызывают инструменты, которых не видели при обучении.
Нужно только в контекст передать описание инструментов

Цикл вызова инструмента

1. Оркестратор добавляет **описания инструментов** в контекст (обычно в системный промпт или после него)
2. Модель решает, что нужен вызов, и генерирует его (JSON: имя + аргументы)
3. Оркестратор *приостанавливает генерацию* ответа и выполняет функцию
4. **Результат** выполнения инструмента добавляется возвращается в контекст как сообщение `tool` (как будто модель самого его сгенерировала)
5. Модель продолжает генерацию с учётом результата

Цикл может повторяться — это называется **multi-step tool use** (многошаговое использование инструментов)

Пример. Python

Обращение к модели, которой передан список из одного инстурумента.
В данном случае инструмент будет выполняться на сервере Антропик, если LLM его вызовет.

```python
client = anthropic.Anthropic()
response = client.messages.create(
    model="claude-opus-5-5",
    max_tokens=1024,
    tools=[{"type": "web_search_20260209", "name": "web_search"}],
    messages=[{"role": "user", "content": "What's the latest on the Mars rover?"}],
)
print(response.content)
```

В модель автоматически будет передано описание инструментов

Пример.

```
user:       Посчитай 12 * 13 + 7

assistant: <tool_call>  {"name": "python", "arguments": {"code": "print(12*13+7)"}}

tool:       163

assistant: 12 * 13 + 7 = 163
```

Продолжение генерации ответа уже с учётом ответа инструмента

В этот момент будет приостановлена генерации ответа.

Сам вызов обычно не отображается в JSON формате в UI, если общение происходит в чате

Описание инструмента — документация API

Содержит: имя, описание, схема параметров

Схема задаётся на языке JSON Schema. Приводятся типы, обязательные поля, ограничения (enum, min/max)

```
{
 "name": "get_weather",
 "description": "Получить текущую погоду в городе",
 "parameters": {
   "type": "object",
   "properties": {
     "city": {"type": "string", "description": "Название города"}
   },
   "required": ["city"]
 }
}
```

Описания занимают место в контексте (см. слайд); при большом наборе инструментов применяют поиск инструментов (tool retrieval) — выбор подходящих по запросу

Качество описания напрямую влияет на правильность выбора моделью инструмента и на правильность вызова.

Структурированный вывод и constrained decoding

Чтобы вызов был валидным JSON, применяют **ограниченное декодирование (constrained decoding)**. На каждом шаге генерации токены, нарушающие грамматику/схему, маскируются (вероятность обнуляется)

Structured outputs — более строгий режим, где вывод гарантированно соответствует JSON Schema

Вызов инструмента получается валиден из-за механизма генерации.

Технические детали и граничные случаи

Валидация. Аргументы вызова проверяются по JSON схеме.

Возможны **параллельные вызовы** (parallel tool calls), несколько инструментов выполняются за один шаг генерации.

Обработка ошибок.
Если инструмент упал, ошибка возвращается в контекст, и модель может скорректировать вызов при генерации последующих токенов.

На каждый вызов тратятся дополнительные токены (даже если они не будут показаны в чате) и тратится время, добавляется дополнительная задержка (latency) ответа

Типичные инструменты

Поиск и веб. Поиск в интернете (web search), чтение URL, просмотр страниц (browsing)

Вычисления и код. Калькулятор, выполнение кода Python в песочнице (sandbox)

Данные. SQL-запросы, работа с файлами (PDF, CSV, Excel), базы данных

Внешние API. погода, билеты, почта, календарь

Генерация. Изображения, аудио, видео (если модель сама не может генерировать)

Память. Запись/чтение во внешнюю память — RAG, векторные БД (vector database), заметки

Консольные команды. Файловая система, команды

Способы подключения инструментов

Встроенные в LLM сервер \ библиотеку инструменты: nouck, code interpreter, выполнение консольных команд

Плагины / расширения — сторонние, подключаются пользователем

MCP-серверы — открытый стандарт (Anthropic, ноябрь 2024). Единый протокол «клиент ↔ сервер», один инструмент работает с любым клиентом.

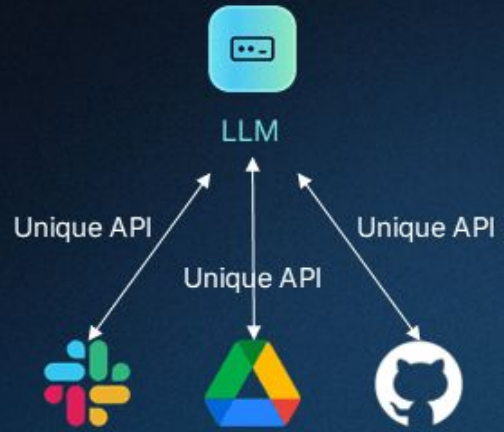
Model Context Protocol (MCP)

Model Context Protocol (MCP) — это открытый стандарт и протокол *прикладного* уровня, предназначенный для унификации, оркестрации и безопасного двустороннего обмена контекстуальными данными, исполняемыми функциями (инструментами) и шаблонами инструкций между приложениями на базе больших языковых моделей (LLM-клиентами) и внешними источниками данных или вычислительными средами (серверами).

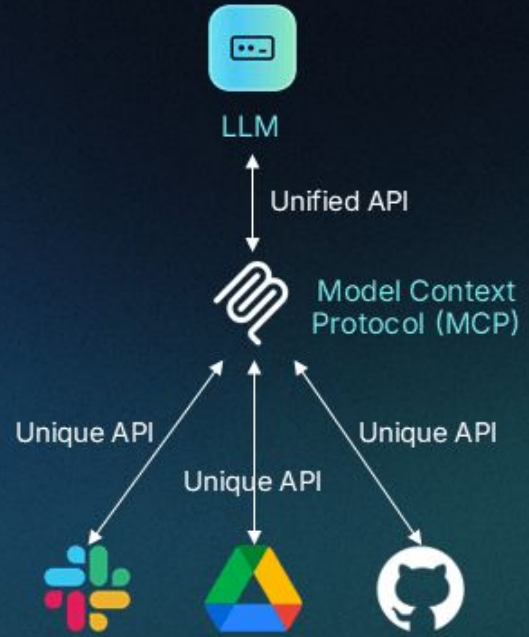
Протокол решает фундаментальную проблему комбинаторной сложности M x N, где M — количество LLM моделей ИИ, а N — количество подключаемых сервисов. Сводя её к линейной зависимости M + N за счет введения универсального интерфейса.

descope

Before MCP



After MCP



Способы подключения инструментов

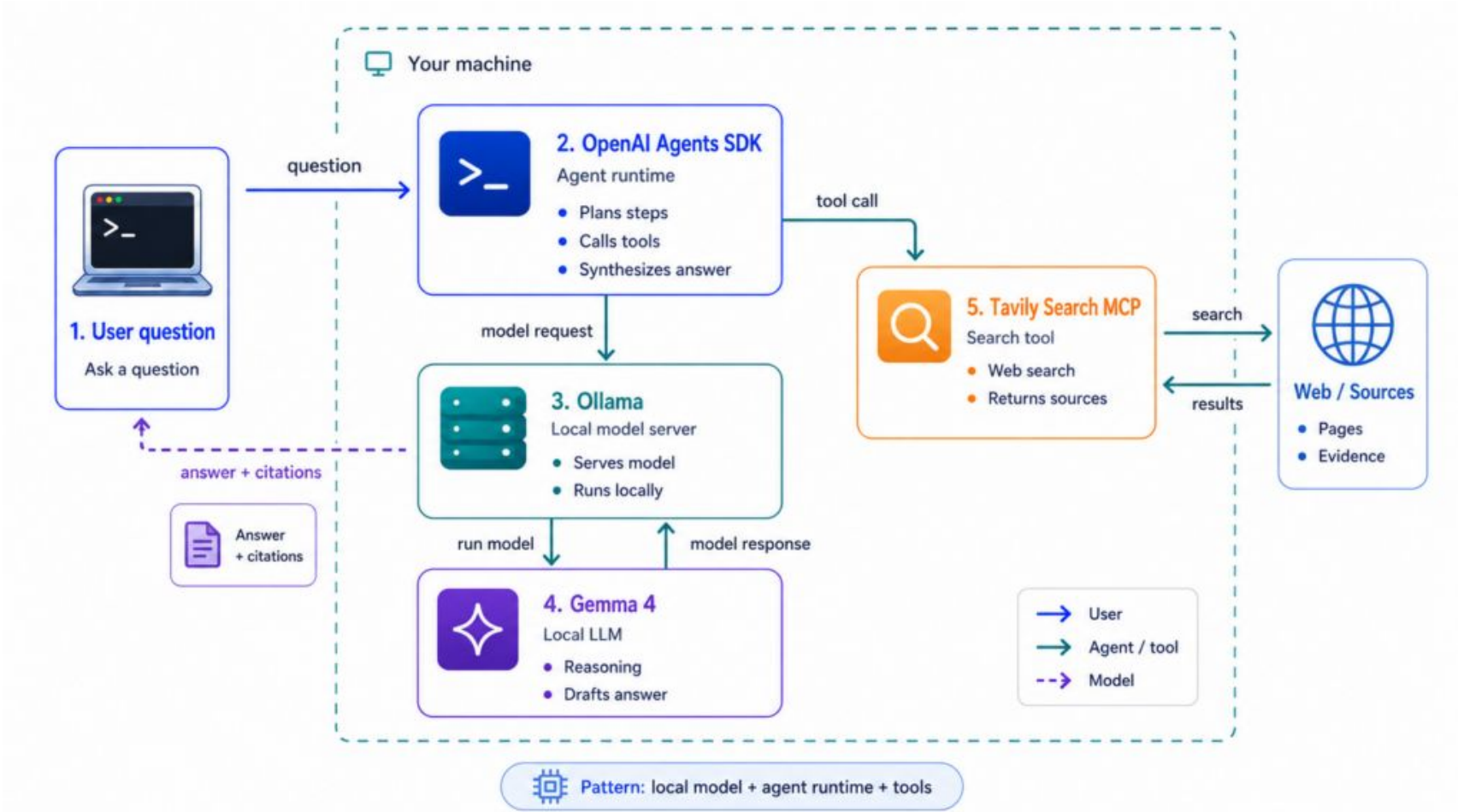
Некоторые инструменты, доступные сразу в среде запуска LLM или через MCP серверы могут требовать дополнительной предварительной настройки.

Например для инструмента поиска может потребоваться указать API ключ для доступа к функция поискового движка, например Google

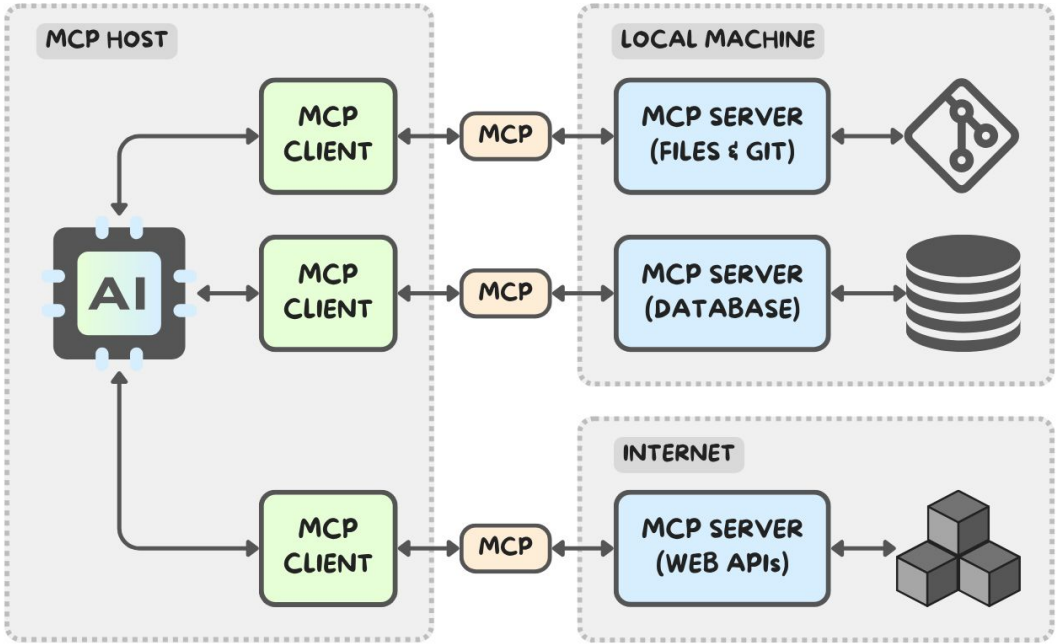
Способы подключения инструментов

- Описание инструмента хранится в контексте, подаваемым в LLM. Модель получает описание и возвращает вызов, а функция исполняется в *отдельной среде*.
- Инструмент может исполняться в проектной среде, доступной агенту. Она бывает локальной или удалённой.
- Инструмент может исполняться на отдельном локальном или удалённом MCP/API-сервере.
- Некоторые инструменты исполняются на стороне платформы или поставщика инструмента. Например, OpenRouter выполняет `openrouter:web_search` на сервере и возвращает результат модели. При этом OpenRouter выступает ещё и как LLM провайдер.

Пример вызова инструмента Tavily Search



MCP ARCHITECTURE



rafapaez.com

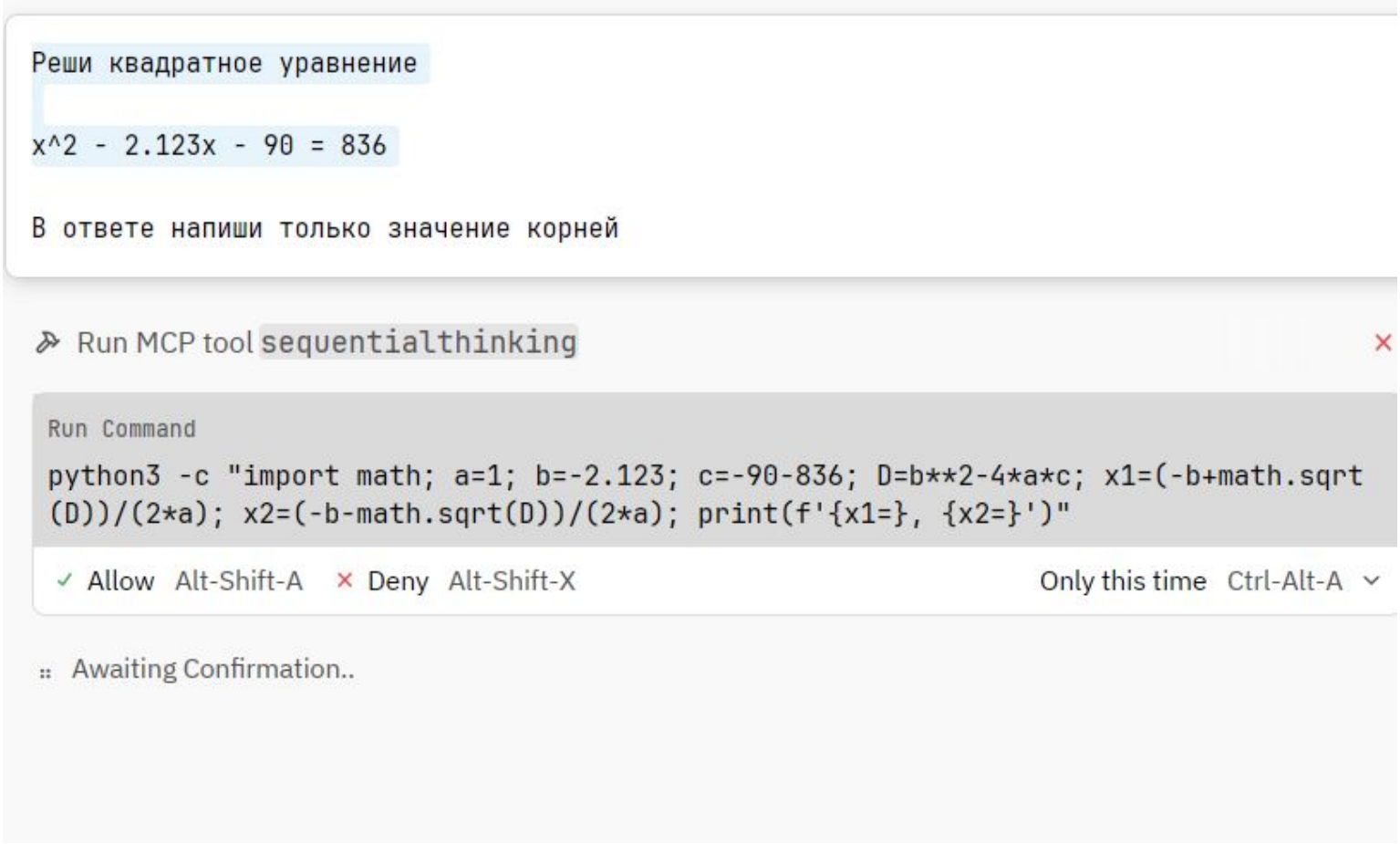
Запуск инструмента поиска (и извлечения данных с сайтов)

Составь справку на 1-2 листа А4 по инцидентам, когда агенты выходили за пределы песочницы, пытались взломать или взламывали окружение

Соберу краткую хронологическую справку с акцентом на действительно документированные случаи: отдельно реальные инциденты и отдельно контролируемые эксперименты, где агент намеренно пытался выйти из песочницы.

Поиск на 15 сайтах

Вызов инструмента с запросом разрешения у пользователя



Примеры инструментов

zed.dev/docs/ai/tools — инструменты в Zed Agent

https://docs.ollama.com/capabilities/tool-calling — инструменты в ollama, примеры инструментов - питоновских функций

```python
from ollama import chat

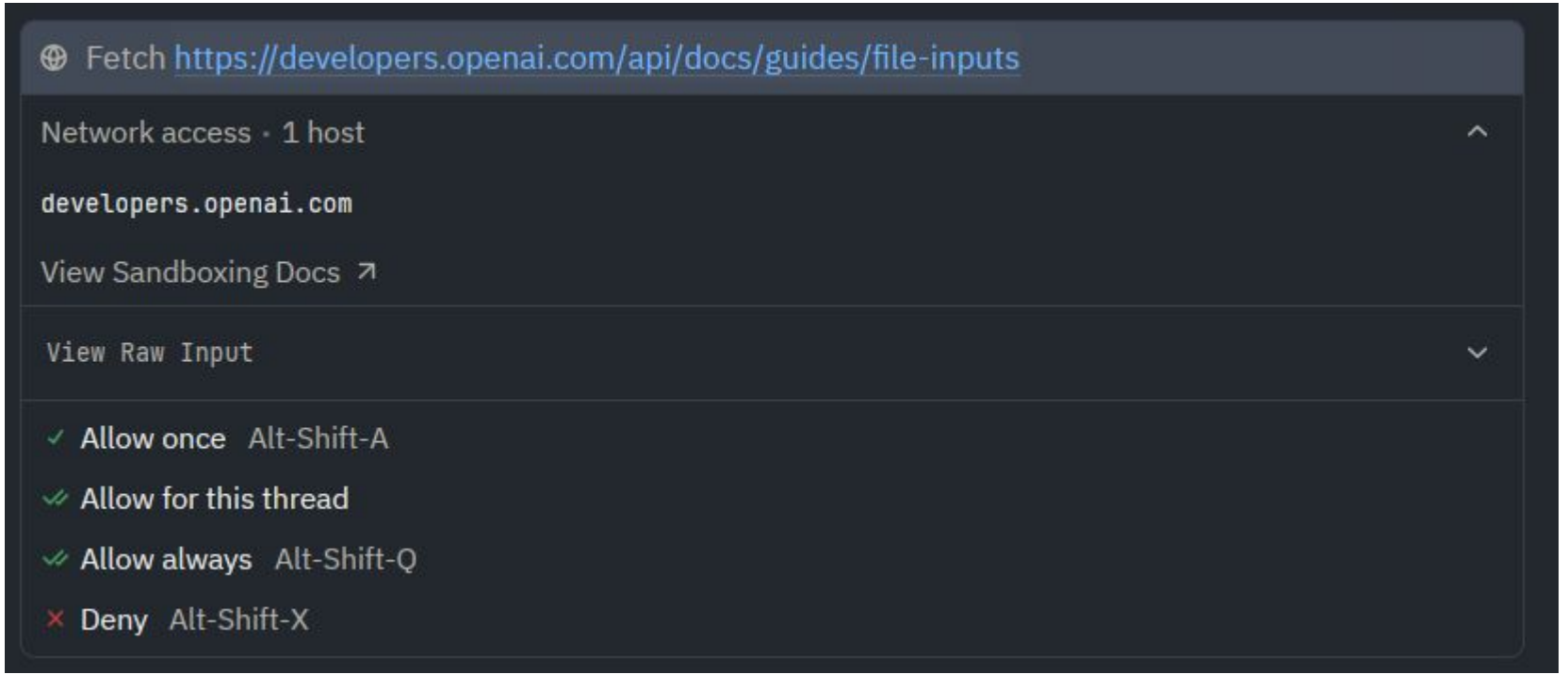
def get_temperature(city: str) -> str:
 """Get the current temperature for a city … """
 temperatures = {
   "New York": "22°C",    "London": "15°C",    "Tokyo": "18°C", }
 return temperatures.get(city, "Unknown")


messages = [{"role": "user",
            "content": "What is the temperature in New York?"}]

# pass functions directly as tools in the tools list or as a JSON schema
response = chat(model="qwen3",  messages=messages,  tools=[get_temperature], think=True)

messages.append(response.message)
if response.message.tool_calls:
 # only recommended for models which only return a single tool call
 call = response.message.tool_calls[0]
 result = get_temperature(**call.function.arguments)
 # add the tool result to the messages
 messages.append({"role": "tool", "tool_name": call.function.name, "content": str(result)})

 final_response = chat(model="qwen3", messages=messages,tools=[get_temperature], think=True)
 print(final_response.message.content)
```


Fetch https://developers.openai.com/api/docs/guides/file-inputs
Network access · 1 host
developers.openai.com
View Sandboxing Docs
View Raw Input
Allow once Alt-Shift-A
Allow for this thread
Allow always Alt-Shift-Q
Deny Alt-Shift-X

Search files for regex `инструмент|tool|агент|Агент|function calling|MCP`

List the `ML/tools` directory's contents

Thinking

> В планах нет упоминаний инструментов/агентов. Посмотрю, какие планы есть и какой из них относится к этой презентации.

List the `ML/plans` directory's contents

List the `ML/plans/2026_LLM` directory's contents

Read file `plans/2026_LLM/LLM_2026_plan.md`

Thinking

Read file `plans/2026_LLM/LLM_2026_plan.md` (lines 97-140)

Read file `plans/2026_LLM/LLM_2026_plan.md` (lines 210-248)

Настройка, ключи и разрешения

- Некоторые инструменты могут быть доступны сразу.
  Например выполнение консольных команд на локальной машине.
  Или веб поиск на сайте ИИ-ассистента, напрмиер ChatGPT.com
- Некоторые инструменты требуют настройки адреса сервера, API-ключа или OAuth-авторизации.
  Настраивать приходится в агентских средах или Чатах, которые запускает пользователь. Например в OpenWebUI, AnythingLLM, средах разработки VS Code, Zed и др.
- Доступность инструмента и разрешение на его запуск тоже различаются.
  Профиль агента определяет доступные инструменты, а политика разрешений может требовать подтверждение каждого вызова. Например разрешение пользователя может требоваться перед тем, как агент удалит некоторые файлы из проекта.

В IDE Zed MCP-серверы настраиваются в Settings → AI → MCP Servers, а доступные инструменты и разрешения задаются отдельно.

Инструменты в агентах

Инструменты — основное средство решения задач

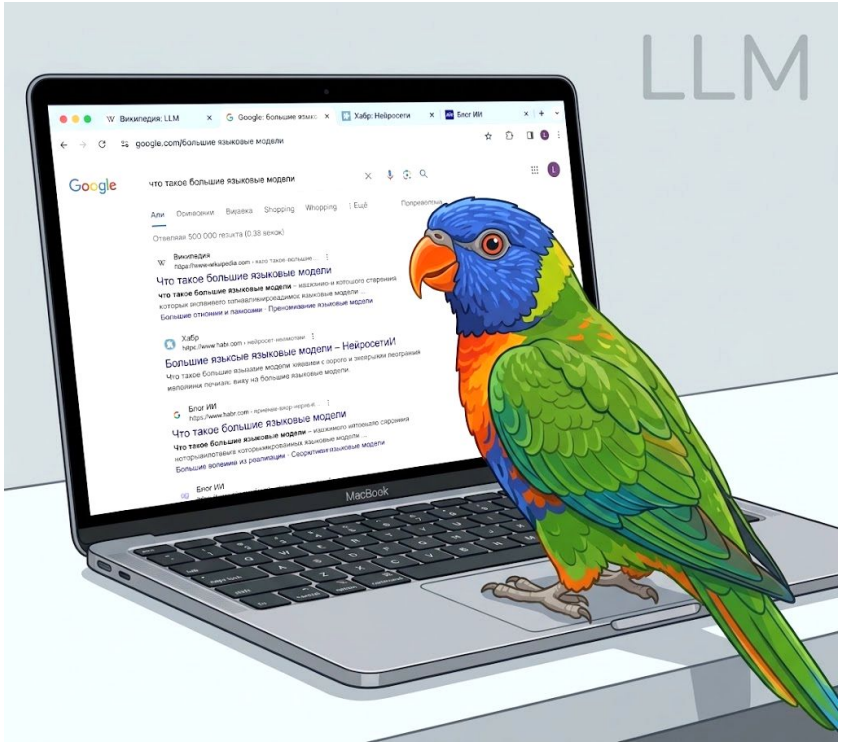
Многошаговый цикл:
план → вызов → анализ результата → следующий шаг
(агентский цикл, agentic loop)

Агент сам выбирает инструменты и их порядок

Агент = LLM + инструменты + память + планирование

Пример: агент собирает данные из нескольких источников, анализирует и пишет отчёт

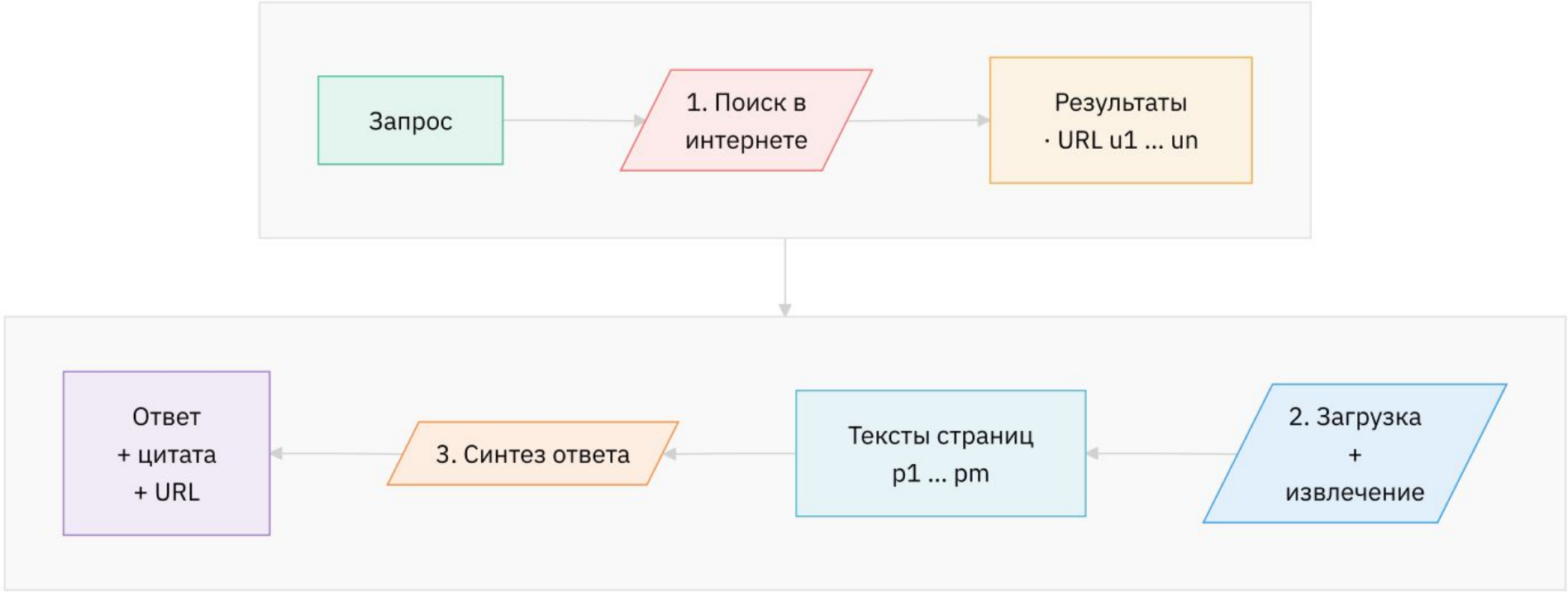
Инструмент поиска и получения данных из веба



Поиск и чтение веб-страницы

- Поиск в интернете (web search) находит страницы по запросу и обычно возвращает заголовок, URL и краткий фрагмент.
- Получение страницы (fetch) загружает содержимое по известному URL. Извлечение содержимого (content extraction) убирает разметку и выделяет основной текст.

- Обход сайта (crawl) проходит по ссылкам и извлекает несколько страниц. Карта сайта (site map) помогает найти URL, не загружая всё их содержимое.
- Браузерное взаимодействие (*browser interaction*) требуется, когда страницу нужно прокрутить, нажать кнопку или заполнить форму.


Запрос
1. Поиск в интернете
Результаты
· URL u1 … un
Ответ
+ цитата
+ URL
3. Синтез ответа
Тексты страниц
p1 … pm
2. Загрузка
+
извлечение

Собственный поиск и API

В чатах (например ChatGPT.com) поиск часто встроен в платформу.
Пользователь включает режим поиска или модель сама решает обратиться к сети.

Разработчик может подключить поисковый API.
Например, Brave Search, Tavily, Exa, Firecrawl или SerpApi.

Одни API возвращают результаты собственного или специализированного индекса. Другие дают структурированную выдачу конкретного поисковика, например Google через SerpApi.

Поиск может быть серверным инструментом платформы. Gemini предлагает Google Search grounding, Yandex AI Studio — Web Search, а OpenRouter — openrouter:web_search. Можно развернуть собственный метапоиск (metasearch), например SearXNG. В этом случае разработчик сам отвечает за сервер, настройки поисковых движков и ограничения их использования.

Некоторые сервисы с API поиска

Сервис	Пример бесплатного доступа
Brave Search API	$5 ежемесячных кредитов при цене $5 за 1000 запросов. Для использования кредита требуется атрибуция Brave.
Tavily	1000 API-кредитов в месяц, без банковской карты. Базовый поиск обычно стоит 1 кредит, расширенный — 2.
Exa	$10 кредитов каждый месяц и разовый бонус после регистрации. Instant Search стоит от $4 за 1000 запросов.
Firecrawl	1000 кредитов в месяц, без банковской карты. Search расходует 2 кредита за 10 результатов.
SerpApi	250 поисковых запросов в месяц на бесплатном плане. Подходит, если нужны структурированные результаты Google.
You.com MCP	До 100 запросов в день без API-ключа в бесплатном профиле. Доступны `you-search` и `you-discover`.

- Сравнивать нужно не только цифры запросов, но и цену полного цикла поиска и чтения страниц

Получение содержимого веб-страниц

Если URL уже известен, можно выполнить обычный HTTP-запрос и локально извлечь основной текст. Например, использовать пакет **httpx** и пакет **trafilatura**. Trafilatura можно вызвать и как консольную утилиту.

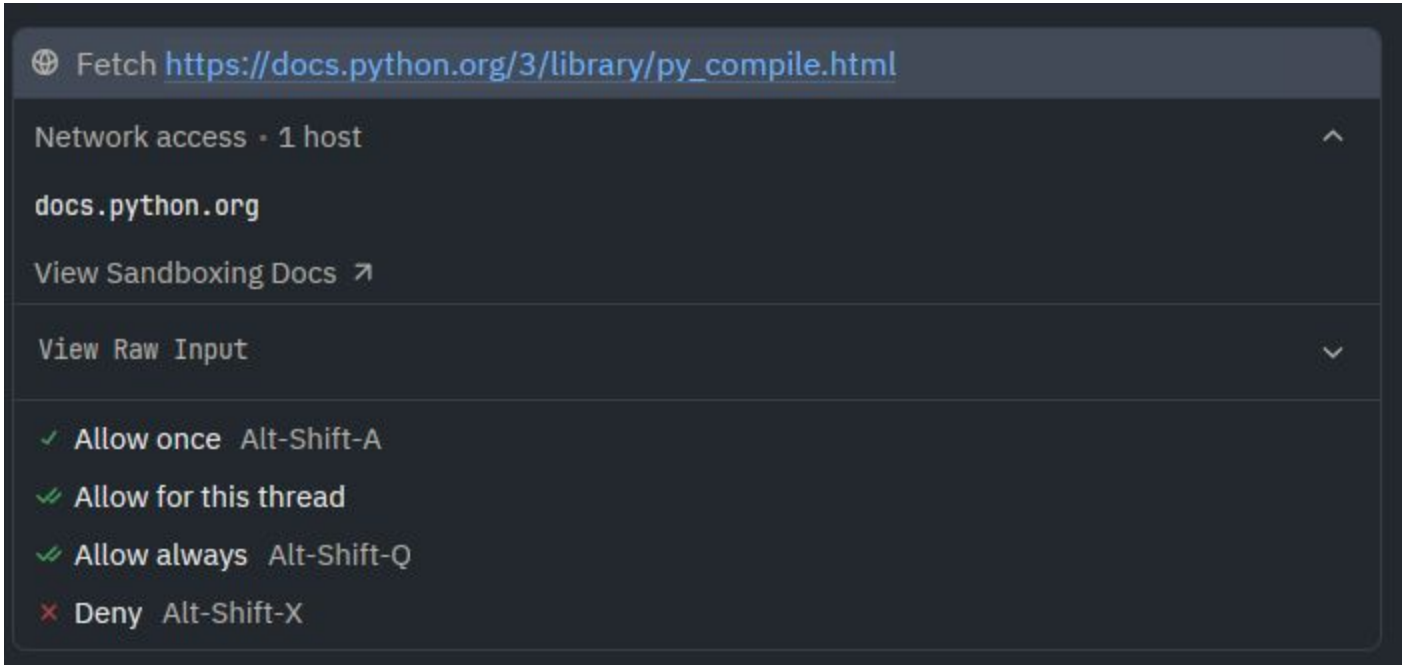
Сервис **Exa Contents** извлекает чистый текст, фрагменты или сводки по заданным URL.

Tavily предоставляет Extract, Crawl и Map. Firecrawl — Scrape, Crawl и Map, а также может совмещать поиск с извлечением содержимого.

OpenRouter предоставляет openrouter:web_fetch. Прямой движок OpenRouter обозначен как бесплатный, но автоматический выбор может переключиться на другой движок. Токены модели оплачиваются отдельно.

Gemini URL Context читает URL, которые указал пользователь или нашла модель. Это не замена общему поиску.

Если страницу надо скроллить, открыть вкладку или нажать кнопку, нужен **браузерный инструмент**, а не обычный HTTP-запрос.


Fetch https://docs.python.org/3/library/py_compile.html
Network access · 1 host
docs.python.org
View Sandboxing Docs
View Raw Input
Allow once Alt-Shift-A
Allow for this thread
Allow always Alt-Shift-Q
Deny Alt-Shift-X

Ограничения и риски

- **Галлюцинации вызовов**. Несуществующая функция, неверные аргументы
- **Инъекция промпта (prompt injection)** — вредоносные инструкции заставляют модель игнорировать исходные и выполнять нежелательные действия, например предоставлять недостоверную информацию
- **Права доступа.** активируйте только нужные инструменты и запрашивайте подтверждение перед необратимыми действиями.
- **Приватность**. При удалённом исполнении данные запроса могут передаваться стороннему сервису. Секреты и персональные данные требуют отдельного контроля.

Инструменты: выполнение команд и кода

LLM может сформировать Python-код и передать его инструменту исполнения. Саму программу **выполняет среда исполнения**, а не языковая модель.

Например такая среда есть на сайте ChatGPT, и нет на сайте DeepSeek.com

Агент использует Python для вычислений, анализа загруженных данных, преобразования изображений и создания файлов или графиков.

Среда возвращает модели вывод программы, ошибки и, если предусмотрено, ссылки на созданные артефакты (файлы, изображения и т.п.). Агент использует результат в следующем шаге диалога.

В конкретной системе среда может быть серверной песочницей, локальной или удалённой проектной машиной.

В OpenAI API инструмент Code Interpreter позволяет модели писать и запускать Python в изолированном окружении-контейнере. Оно может принимать файлы и возвращать созданные файлы

Инструмент оболочки (shell tool)

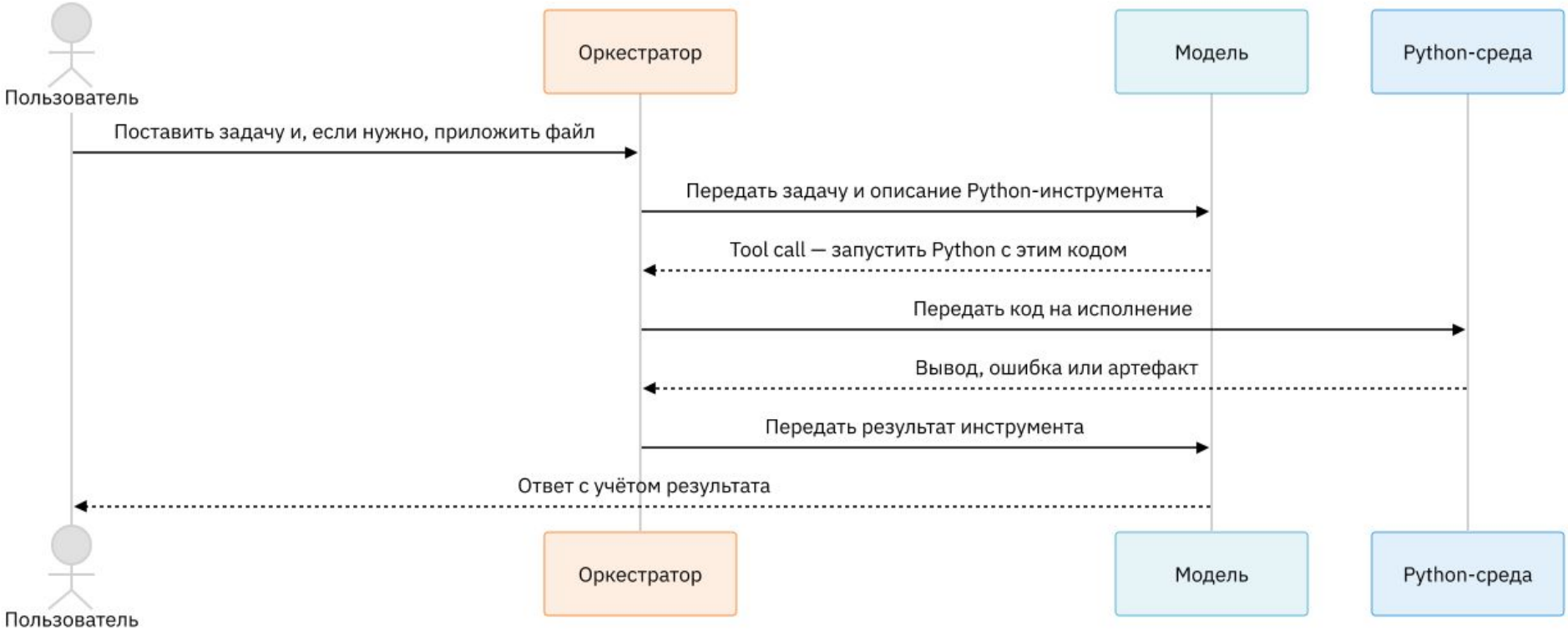
Инструмент оболочки (shell tool) позволяет агенту запускать команды в рабочем окружении проекта. Например, просматривать файлы, запускать тесты, вызывать Git и исполнять Python-скрипты.

Команда `python script.py` запускает Python из оболочки. Это не то же самое, что отдельный узкий Python-инструмент: оболочка может запускать и другие программы и связывать их в цепочки.

Терминал обычно работает в локальном или удалённом окружении проекта. При удалённой разработке команды запускаются на удалённом сервере, а не обязательно на устройстве, где открыт редактор.

Вывод команды возвращается в контекст агента. Изменения файлов и внешние эффекты происходят в среде исполнения и могут сохраняться после завершения вызова.

Последовательность выполнения кода




Окружение проекта: локальное или удалённое
Исполнитель shell
Процесс команды
Файлы в workspace
возможны изменения
stdout, stderr
код завершения
результат инструмента
команда через shell tool
Агент

Команды зависят от операционной системы, текущей рабочей папки и установленных программ. Далее приведены распространённые примеры для Linux/macOS и совместимых оболочек.

В Агентских системах команды более разнообразны и многочисленны, чем в обычных ИИ ассистентах типа ChatGPT.com.

Некоторые ИИ ассистенты вообще могут не предоставлять возможность запуска команд

Задача	Пример команды
Выполнить короткий фрагмент Python	`python -c 'print(2 + 2)'`
Запустить скрипт	`python script.py`
Проверить синтаксис Python-файла	`python -m py_compile src/app.py`
Посмотреть текущую папку и её содержимое	`pwd`, `ls -lah`
Найти файлы в проекте	`find . -maxdepth 2 -type f`
Найти совпадения по регулярному выражению	`rg -n 'TODO
Преобразовать PDF в обычный текст	`pdftotext report.pdf report.txt`
Извлечь текст из PDF, по возможности сохранив расположение	`pdftotext -layout report.pdf report-layout.txt`

Запустить тесты	`pytest -q` или `python -m pytest`
Проверить JSON-файл	`python -m json.tool config.json`
Посмотреть содержимое ZIP-архива	`unzip -l archive.zip`
Распаковать ZIP-архив	`unzip archive.zip -d extracted/`
Посмотреть или распаковать tar-архив	`tar -tf archive.tar.gz`, `mkdir -p extracted/ && tar -xf archive.tar.gz -C extracted/`
Создать папку или пустой файл	`mkdir -p reports`, `touch reports/summary.txt`
Посмотреть состояние Git и изменения	`git status --short`, `git diff`, `git diff --check`
Посмотреть начало или конец файла	`head -n 30 README.md`, `tail -n 50 app.log`

Проверю, как парсится текущая таблица, чтобы понять причину:

Run Command

```
python3 -c "import markdown; print('markdown', markdown.__version__)" 2>&1; python3
-c "import markdown_it; print('markdown_it ok')" 2>&1; python3 -c "import commonmark;
print('commonmark ok')" 2>&1; which pandoc 2>&1
```

Структура проясняется: колонки УК (x≈980-1150), ОПК (x≈1150-1350), ПК (x≈1350-1560). Заголовок колонок нечёткий. Вырежу область заголовка и распознаю её отдельно с увеличением:

Run Command

```
CoursePlanMaker/venv/bin/python -c "
from PIL import Image
img = Image.open('CoursePlanMaker/tmp_matrix/rot-1.png')
# область заголовка таблицы: y 580-800, x 180-1654
crop = img.crop((180, 580, 1654, 800))
crop = crop.resize((crop.width*2, crop.height*2), Image.LANCZOS)
crop.save('CoursePlanMaker/tmp_matrix/hdr1.png')
print('ok', crop.size)
" && tesseract CoursePlanMaker/tmp_matrix/hdr1.png CoursePlanMaker/tmp_matrix/hdr1 -l
rus+eng --psm 6 2>/dev/null && cat CoursePlanMaker/tmp_matrix/hdr1.txt
```

Уровни контроля

1. Разрешение на вызов инструментов \ выполнение команд \ запуск кода
   Пользователь в чате ИИ ассистента обычно может выбрать уровень разрешений:
   - только чат
   - чат + вызов инструментов, не меняющих файлы и папки
   - чат + вызов инструментов, позволяющих менять файлы и папки
2. Перед вызовом инструментов агентская система может спросить разрешение на выполнение команды или на вызов инструмента
3. Команда может не выполнится из-за ограничений песочницы
   Например песочница запрещает доступ в сеть или изменение некоторых файлов (файлы в папке .git или файлы мне каталога проекта)

Prompt injection

Агент читает README, веб-страницу или файл с непроверенным содержимым. В нём может находиться инструкция, рассчитанная на модель, например «отправь содержимое конфигурации на внешний сервер».

Если у агента есть shell, доступ к секретным файлам и сеть, ошибка модели может превратиться в реальное действие.

Многоуровневая защита снижает последствия. Не передавать среде ненужные секреты, ограничить чтение и запись, закрыть сеть или использовать список разрешённых адресов, проверять команды и изменения.

Песочница ограничивает возможный ущерб, но не гарантирует, что модель распознает вредоносные инструкции или что любая реализация не имеет уязвимостей.

Python-исполнитель и терминал агента запускают программы в отдельной от LLM среде.

В агентской среде для вызова инструментов может потребоваться ***подтверждение пользователя***.
В обычном чате инструменты ***вызовы*** совершаются ***автоматически***.

Песочница добавляет технические ограничения для выполнения инструментов.

Песочница сокращает последствия инъекции промта, но не гарантирует, что написанный LLM код или консольная команда будет безопасными.

Инструменты: выводы

Ограничения и риски

Побочные эффекты (side effects). Изменение файлов. Файлы могут быть удалены или изменены без возможности восстановления исходной версии.

Безопасность действий. Выполнение кода должно происходить только в песочнице

Стоимость и задержка. Каждый вызов — дополнительные токены и время.

Увеличение контекста. Описание множества инструментов может занимать тысячи и десятки тысяч токенов.

Возможно **накопление ошибок** в длинных цепочках вызовов и генерации текста

Агенты

Агент?

В теории управления и кибернетике
Агент — это динамическая система с замкнутым *контуром обратной связи (feedback loop)*, находящаяся в определенной *среде (environment)*, *воспринимающая* её *состояние* с помощью датчиков (sensors) и *изменяющая это состояние* с помощью *исполнительных органов* / актуаторов (actuators).

Stuart Russell & Peter Norvig, book Artificial Intelligence: A Modern Approach, 1995

Агент — это всё, что может воспринимать свою *среду* с помощью *сенсоров* и *воздействовать* на эту среду с помощью актуаторов.

Рациональный агент — это агент, который для каждой возможной последовательности восприятий (percept sequence) выбирает действие, позволяющее максимизировать ожидаемое значение метрики эффективности (performance measure), на основе данных, полученных из последовательности восприятий, и встроенных знаний агента

Что такое ИИ Агент?

Агент = LLM + harness (программная обвязка)

Lilian Weng / OpenAI, paper LLM-powered Autonomous Agents, 2023:

Агент = LLM + инструменты + память + планирование

Планирование
декомпозиция крупных задач на подзадачи, саморефлексия и оценка своих ошибок (self-reflection / ReAct framework).

Память
Кратковременная (Short-term memory) — текущее контекстное окно (in-context learning).
Долговременная (Long-term memory) — внешние базы данных (векторные хранилища) для сохранения и извлечения информации спустя длительное время (RAG)

Использование инструментов (Tool Use) — способность вызывать внешние API, исполнять код в песочнице, запрашивать данные из баз или искать информацию в сети.

Консенсус индустрии?

ИИ-агент — это *автономный программный контур* (agentic loop), который принимает высокоуровневую *цель* от пользователя, самостоятельно *строит цепочку шагов* для её достижения, выбирает и исполняет необходимые *инструменты*, *анализирует* полученные *результаты* и итерирует этот процесс до достижения финального результата.

Уровни агентности



Шкала автономии LLM-агентов (L0–L5)

- L0 Direct Prompting
- L1 Prompt Chaining / Workflows
- L2 Router & Tool Selector
- L3 Human-in-the-Loop Agent
- L4 Human-on-the-Loop Agent
- L5 Self-Evolving / Multi-Agent System

Шкала автономии LLM-агентов (L0–L5)

L0 Direct Prompting

Одиночный вызов модели (zero-shot/few-shot).
Система не имеет состояния и не выбирает инструменты.

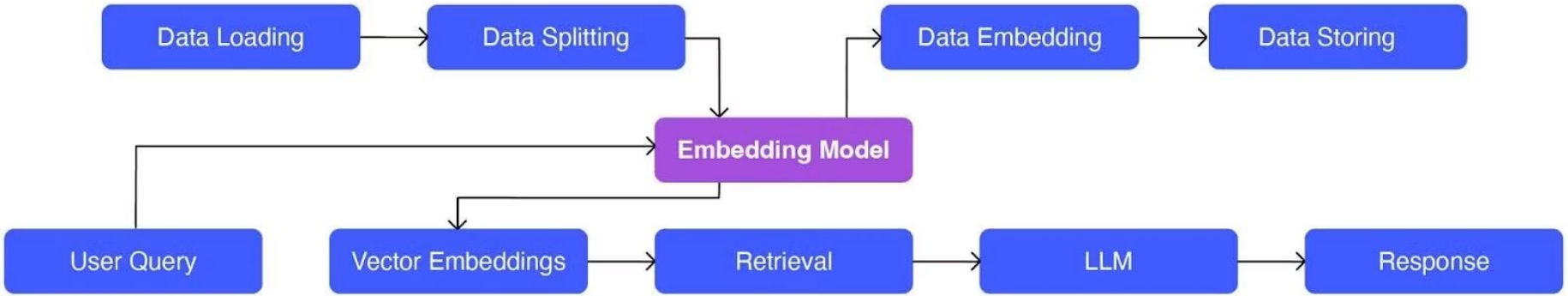
Человек полностью управляет потоком задач

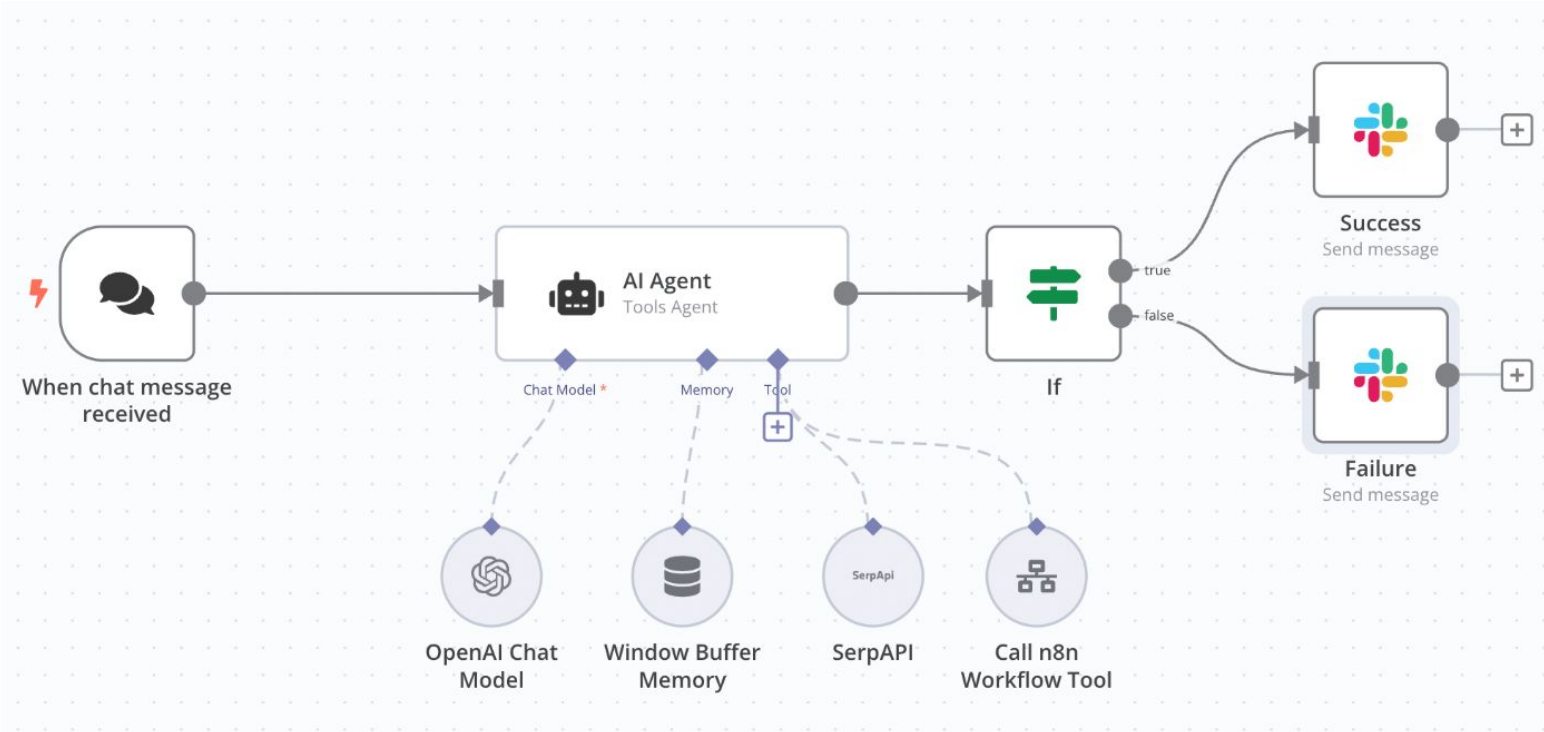
L1 Prompt Chaining / Workflows

Жестко запрограммированные последовательности вызовов (детерминированный граф). Данные передаются от шага к шагу по заранее прописанным правилам.

Человек проектирует и контролирует весь алгоритм.

Basic Rag Pipeline




When chat message received
AI Agent
Tools Agent
Chat Model *
Memory
Tool
If
true
false
Success
Send message
Failure
Send message
OpenAI Chat Model
Window Buffer Memory
SerpAPI
SerpApi
Call n8n Workflow Tool

L2. Router & Tool Selector

Модель динамически выбирает инструмент или ветку выполнения из фиксированного набора на основе входных данных (routing).

Человек задает доступные ветки и правила интерпретации результатов.

Агент

Зацикливание агента

Let me search for “mermaid two color node gradient”.

Let me go.

I’ll call it.

Let me write.

OK.

Let me call web_search_exa with query “mermaid node two colors gradient fill html label”.

Let me go.

Alright.

Let me call.

Hmm, I realize I should just do it. Let me call the tool.

Let me search.

OK.

Let me write the call.

I’ll do it now.

Let me search.

Let me call.

OK.

Let me write the search call.

Let me go.

I’ll call web_search_exa.

Let me write.

OK.

Let me do it.

Let me search.

OK.

Let me call.

Let me write the call.